ANO 02 | FASE 03 | CAP 9 | ATIVIDADE 02 | REUSE

Alunos:
Natali Schers
Rebeca Luiza Soares Cerqueira
Sarah Raquel Soares Maranhão
Stephanie da Silva Cruz

Link do GitHub:
https://github.com/web-design-on/reuse/tree/main

01. Consumo de APIs:

API Utilizada:

- O aplicativo consome a API DummyJSON para buscar os produtos, categorias e detalhes de cada item. Além disso, a mesma API foi utilizada para a simulação do fluxo de login, utilizando a base de usuários e o end point disponibilizados para testes de autenticação.

Telas Atualizadas:

- **Home** (`index.tsx`): Recebe e lista os produtos gerais em destaque.
- **Favoritos** (`favorites.tsx`): Consome a lista de produtos (através do hook `useProducts`) para extrair dinamicamente a lista de categorias disponíveis e filtrar os produtos exibidos de acordo com a categoria selecionada pelo usuário. O idioma das categorias foi traduzido manualmente de inglês para português.
- **Detalhes do Produto** (`details.tsx`): Recebe os dados de um produto específico via parâmetros de rota para exibir informações detalhadas, como:
    - imagens em carrossel;
    - descrição;
    - dimensões;
    - disponibilidade;
    - tags;
    - marca;
    - preço.
- **Login** (`login.tsx`): autentica o usuário a partir do usuário e senha (“username” e “password”, considerando as propriedades da API).
- **Perfil** (`profile.tsx`): a partir dos dados salvos na sessão, atualiza as informações do usuário, como nome e foto de perfil retornados pela API. Caso não encontre o usuário, faz o redirect para a tela de login. Caso encontre, disponibiliza um botão de “logout” que limpa os tokens de acesso e dados da sessão.

02. Autenticação e Sessões:

- O login é feito pelo endpoint "https://dummyjson.com/auth/login", que espera receber um usuário e uma senha. Caso a autenticação seja bem sucedida, os dados retornados, bem como os tokens de acesso e "refresh", são armazenados em SecureStore e o estado de "user" é atualizado no context.

  Ao acessar a tela de perfil, buscamos os dados do usuário utilizando o context e disponibilizamos as informações na tela. Além disso, um auto-refresh é feito a cada 25 minutos (antes da expiração do token que acontece a cada 30 minutos), os tokens são renovados automaticamente e os dados do usuário são atualizados, garantindo que informações estejam sempre atualizadas.

  Caso o usuário escolha sair, limpamos a sessão, apagando os tokens de acesso e os dados de user.

03. Caching Local (Armazenamento Assíncrono)

O projeto utiliza o `AsyncStorage` (`@react-native-async-storage/async-storage`) para persistir dados localmente no dispositivo do usuário, garantindo que as informações não sejam perdidas ao fechar o app.

Itens Armazenados em Cache e Telas Aplicadas:

- **Favoritos (`@reuse_favs`):**
  - `details.tsx`: Permite salvar ou remover produtos da lista de favoritos através do botão “Favoritar”, utilizando a função toggleFavorite, com atualização dinâmica da interface.
  - `favorites.tsx`: Recupera os itens armazenados localmente através da função loadSavedItems, permitindo a exibição offline dos produtos favoritados.
- **Carrinho de Compras (`@reuse_cart`):**
  - `details.tsx`: Permite adicionar ou remover produtos do carrinho utilizando a função toggleCart, salvando os dados localmente no cache. O estado do botão é atualizado em tempo real conforme a presença do item no armazenamento.